

ASSIGNMENT-11.5

H.no-2303A51927

Batch – 30

Task 1 – Stack Implementation

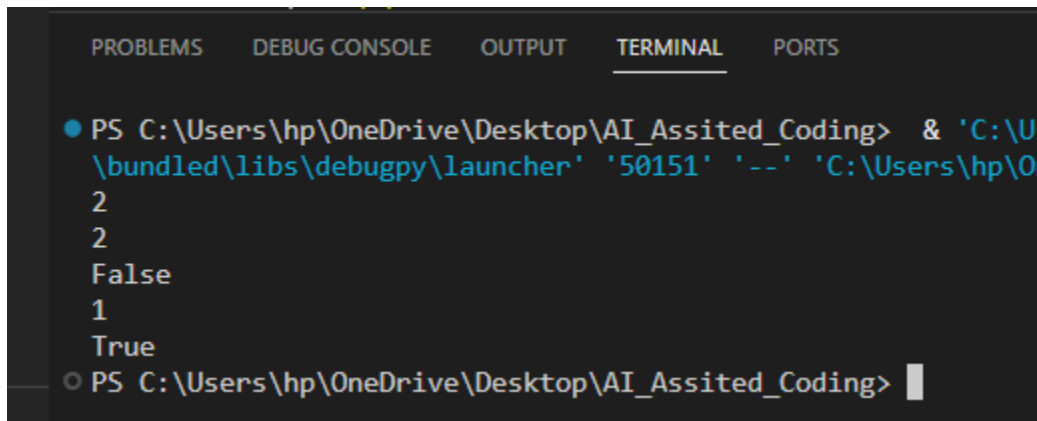
Prompt

Design and implement a Stack data structure supporting basic stack operations. A Python Stack class supporting push, pop, peek, and is_empty operations with proper documentation.

CODE

```
Lab_exam.py  Assignmnet_10.2.py  Assignmnet_13.1.py  Assignment_13.5.py  Assignment_11.5.py X  Assignment_9.5.py
Assignment_11.5.py >...
1  #Design and implement a Stack data structure supporting basic stack operations. A Python Stack class supporting push, po
2  class Stack:
3      """A simple implementation of a Stack data structure."""
4
5      def __init__(self):
6          """Initialize an empty stack."""
7          self.items = [] # Use a list to store stack items
8
9      def push(self, item):
10         """Add an item to the top of the stack."""
11         self.items.append(item) # Append item to the end of the list
12
13     def pop(self):
14         """Remove and return the item at the top of the stack. Raises an error if the stack is empty."""
15         if self.is_empty():
16             raise IndexError("Pop from an empty stack") # Raise error if stack is empty
17         return self.items.pop() # Remove and return the last item in the list
18
19     def peek(self):
20         """Return the item at the top of the stack without removing it. Raises an error if the stack is empty."""
21         if self.is_empty():
22             raise IndexError("Peek from an empty stack") # Raise error if stack is empty
23         return self.items[-1] # Return the last item in the list
24
25     def is_empty(self):
26         """Return True if the stack is empty, False otherwise."""
27         return len(self.items) == 0 # Check if the list is empty
28
29 # Example usage:
30 stack = Stack() # Create a new stack instance
31 stack.push(1) # Push an item onto the stack
32 stack.push(2) # Push another item onto the stack
33 print(stack.peek()) # Output: 2 (the top item)
34 print(stack.pop()) # Output: 2 (removes the top item)
35 print(stack.is_empty()) # Output: False (stack is not empty)
36 print(stack.pop()) # Output: 1 (removes the last item)
37 print(stack.is_empty()) # Output: True (stack is now empty)
```

OUTPUT

A screenshot of a terminal window with tabs for PROBLEMS, DEBUG CONSOLE, OUTPUT, TERMINAL, and PORTS. The TERMINAL tab is active. It shows a PowerShell command being executed: PS C:\Users\hp\OneDrive\Desktop\AI_Assited_Coding> & 'C:\Us... \bundled\libs\debugpy\launcher' '50151' '--' 'C:\Users\hp\On... The output of the command is displayed on the next lines: 2, 2, False, 1, True. The prompt PS C:\Users\hp\OneDrive\Desktop\AI_Assited_Coding> is visible at the bottom.

```
PROBLEMS  DEBUG CONSOLE  OUTPUT  TERMINAL  PORTS

● PS C:\Users\hp\OneDrive\Desktop\AI_Assited_Coding> & 'C:\Us
  \bundled\libs\debugpy\launcher' '50151' '--' 'C:\Users\hp\On
  2
  2
  False
  1
  True
○ PS C:\Users\hp\OneDrive\Desktop\AI_Assited_Coding> 
```

JUSTIFICATION

A stack is a linear data structure that follows the Last In First Out (LIFO) principle. The element inserted last is removed first. Stacks are widely used in function calls, recursion, undo operations, and expression evaluation in compilers.

Task 2 – Queue Implementation

Prompt

Create a Queue data structure following FIFO principles. A complete Queue implementation including enqueue, dequeue, front element access, and size calculation.

CODE

```

39
40 #Create a Queue data structure following FIFO principles. A complete Queue implementation including enqueue, deque
41 class Queue:
42     """A simple implementation of a Queue data structure following FIFO principles."""
43
44     def __init__(self):
45         """Initialize an empty queue."""
46         self.items = [] # Use a list to store queue items
47
48     def enqueue(self, item):
49         """Add an item to the end of the queue."""
50         self.items.append(item) # Append item to the end of the list
51
52     def dequeue(self):
53         """Remove and return the item at the front of the queue. Raises an error if the queue is empty."""
54         if self.is_empty():
55             raise IndexError("Dequeue from an empty queue") # Raise error if queue is empty
56         return self.items.pop(0) # Remove and return the first item in the list
57
58     def front(self):
59         """Return the item at the front of the queue without removing it. Raises an error if the queue is empty."""
60         if self.is_empty():
61             raise IndexError("Front from an empty queue") # Raise error if queue is empty
62         return self.items[0] # Return the first item in the list
63
64     def size(self):
65         """Return the number of items in the queue."""
66         return len(self.items) # Return the length of the list
67
68 # Example usage:
69 queue = Queue() # Create a new queue instance
70 queue.enqueue(1) # Enqueue an item
71 queue.enqueue(2) # Enqueue another item
72 print(queue.front()) # Output: 1 (the front item)
73
74 print(queue.dequeue()) # Output: 1 (removes the front item)
75 print(queue.size()) # Output: 1 (one item left in the queue)
76 print(queue.dequeue()) # Output: 2 (removes the last item)
77 print(queue.is_empty()) # Output: True (queue is now empty)

```

OUTPUT

```

AttributeError: 'Queue' object has no attribute 'is_empty'
● PS C:\Users\hp\OneDrive\Desktop\AI_Assited_Coding> c:; cd 'c:\Users\hp\
.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\d
1
1
1
False
2

```

JUSTIFICATION

A queue follows the First In First Out (FIFO) principle where the first inserted element is removed first. Queues are commonly used in CPU scheduling, printer task management, and network data handling where processing order must be maintained.

Task 3 – Singly Linked List

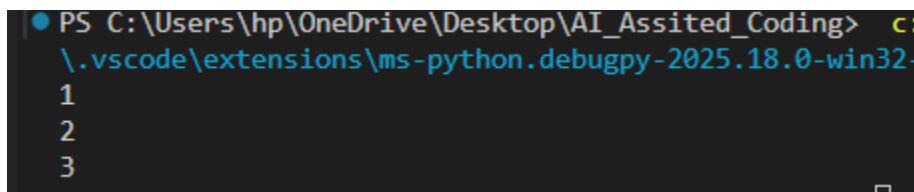
Prompt

Build a singly linked list supporting insertion and traversal. Correctly functioning linked list with node creation, insertion logic, and display functionality.

CODE

```
80
81 #Build a singly linked list supporting insertion and traversal. Correctly functioning linked list with node
82 class Node:
83     """A class representing a node in a singly linked list."""
84     def __init__(self, data):
85         """Initialize a node with data and a pointer to the next node."""
86         self.data = data # Store the data in the node
87         self.next = None # Initialize the next pointer to None
88 class SinglyLinkedList:
89     """A class representing a singly linked list."""
90     def __init__(self):
91         """Initialize an empty linked list."""
92         self.head = None # Initialize the head of the list to None
93
94     def insert(self, data):
95         """Insert a new node with the given data at the end of the list."""
96         new_node = Node(data) # Create a new node with the provided data
97         if self.head is None:
98             self.head = new_node # If the list is empty, set the new node as the head
99             return
100         last_node = self.head # Start from the head of the list
101         while last_node.next: # Traverse to the end of the list
102             last_node = last_node.next
103         last_node.next = new_node # Link the last node to the new node
104
105     def display(self):
106         """Display all nodes in the linked list."""
107         current_node = self.head # Start from the head of the list
108         while current_node: # Traverse through the list until the end
109             print(current_node.data) # Print the data of the current node
110             current_node = current_node.next # Move to the next node
111 # Example usage:
112 linked_list = SinglyLinkedList() # Create a new singly linked list instance
113
114 linked_list.insert(1) # Insert a node with data 1
115 linked_list.insert(2) # Insert a node with data 2
116 linked_list.insert(3) # Insert a node with data 3
117 linked_list.display() # Display the linked list (Output: 1, 2, 3)
118
```

OUTPUT



```
PS C:\Users\hp\OneDrive\Desktop\AI_Assited_Coding> .\vscode\extensions\ms-python.debugpy-2025.18.0-win32-
1
2
3
```

JUSTIFICATION

A singly linked list is a dynamic data structure where elements are stored in nodes connected through pointers. It allows efficient insertion and deletion operations. Linked lists are widely used in implementing stacks, queues, and memory management systems.

Task 4 – Hash Table Implementation

Prompt

Create a hash table with collision handling. Hash table supporting insert, search, and delete operations using chaining.

CODE

```
119
120 #Create a hash table with collision handling. Hash table supporting insert, search, and delete operations
121 class HashTable:
122     """A simple implementation of a hash table with collision handling using chaining."""
123     def __init__(self, size=10):
124         """Initialize the hash table with a specified size."""
125         self.size = size # Set the size of the hash table
126         self.table = [[] for _ in range(size)] # Create a list of empty lists for chaining
127     def _hash(self, key):
128
129         """Generate a hash for the given key."""
130         return hash(key) % self.size # Use built-in hash function and mod by size to get index
131     def insert(self, key, value):
132         """Insert a key-value pair into the hash table."""
133         index = self._hash(key) # Get the index for the key
134         for i, (k, v) in enumerate(self.table[index]): # Check for existing key in the chain
135             if k == key:
136                 self.table[index][i] = (key, value) # Update value if key already exists
137                 return
138         self.table[index].append((key, value)) # Add new key-value pair to the chain
139     def search(self, key):
140         """Search for a value by key in the hash table."""
141         index = self._hash(key) # Get the index for the key
142         for k, v in self.table[index]: # Search through the chain at the index
143             if k == key:
144                 return v # Return the value if key is found
145         return None # Return None if key is not found
146     def delete(self, key):
147         """Delete a key-value pair from the hash table."""
148         index = self._hash(key) # Get the index for the key
149         for i, (k, v) in enumerate(self.table[index]): # Search through the chain at the index
150             if k == key:
151                 del self.table[index][i] # Remove the key-value pair if key is found
152                 return
153 # Example usage:
154 hash_table = HashTable() # Create a new hash table instance
155 hash_table.insert("name", "Alice") # Insert a key-value pair into the hash table
156 hash_table.insert("age", 30) # Insert another key-value pair into the hash table
157 print(hash_table.search("name")) # Output: Alice (search for a key in the hash table)
158 print(hash_table.search("age")) # Output: 30 (search for another key in the hash table)
159 hash_table.delete("name") # Delete a key-value pair from the hash table
160 print(hash_table.search("name")) # Output: None (search for a deleted key in the hash table)
161
162
```

OUTPUT

```

PS C:\Users\hp\OneDrive\Desktop\AI_Assited_Coding> c;;
ugpy\launcher' '62445' '--' 'c:\Users\hp\OneDrive\Desktop\AI_Assited_Coding'
Alice
30
None
PS C:\Users\hp\OneDrive\Desktop\AI_Assited_Coding>

```

JUSTIFICATION

A hash table stores data using key-value mapping, allowing extremely fast data access. The average time complexity for searching is $O(1)$. Hash tables are widely used in databases, dictionaries, caching systems, and indexing.

Task 5 – Graph Representation

Prompt

Implement a graph using an adjacency list representation with methods to add vertices, add edges, and display connections.

CODE

```

161
162
163 #Implement a graph using an adjacency list representation with methods to add vertices, add edges, and display connections
164 class Graph:
165     """A simple implementation of a graph using an adjacency list representation."""
166     def __init__(self):
167         """Initialize an empty graph."""
168         self.graph = {} # Use a dictionary to store the adjacency list
169
170     def add_vertex(self, vertex):
171         """Add a vertex to the graph."""
172         if vertex not in self.graph:
173             self.graph[vertex] = [] # Initialize an empty list for the vertex's edges
174
175     def add_edge(self, vertex1, vertex2):
176         """Add an edge between two vertices in the graph."""
177         if vertex1 in self.graph and vertex2 in self.graph:
178             self.graph[vertex1].append(vertex2) # Add vertex2 to the adjacency list of vertex1
179             self.graph[vertex2].append(vertex1) # Add vertex1 to the adjacency list of vertex2 (for undirected graph)
180
181     def display(self):
182         """Display the connections in the graph."""
183         for vertex, edges in self.graph.items():
184             print(f"vertex: {vertex}, edges: {edges}") # Print each vertex and its connected edges
185
186 # Example usage:
187 graph = Graph() # Create a new graph instance
188 graph.add_vertex("A") # Add vertex A to the graph
189 graph.add_vertex("B") # Add vertex B to the graph
190 graph.add_vertex("C") # Add vertex C to the graph
191 graph.add_edge("A", "B") # Add an edge between vertex A and vertex B
192 graph.add_edge("A", "C") # Add an edge between vertex A and vertex C
193 graph.display() # Display the connections in the graph (Output: A: B, C; B: A; C: A)
194

```

OUTPUT

```
PS C:\Users\hp\OneDrive\Desktop\AI_Assited_Coding> c::;
ugpy\launcher' '59669' '--' 'c:\Users\hp\OneDrive\Deskt
A: B, C
B: A
C: A
PS C:\Users\hp\OneDrive\Desktop\AI_Assited_Coding> |
```

JUSTIFICATION

Graphs are used to represent relationships between objects using vertices and edges. The adjacency list representation is efficient for storing sparse graphs. Graphs are widely used in network routing, social networks, and map navigation systems. and supports scalable validation logic.

Task 6 – Smart Hospital Management System

Prompt

Design a Smart Hospital Management System by selecting appropriate data structures for different hospital operations such as patient check-in, emergency case handling, medical records storage, appointment scheduling, and hospital room navigation.

CODE

```

193
194 #Design a Smart Hospital Management System by selecting appropriate data structures for different hospital operations such as patient c
195 class HospitalManagementSystem:
196
197     """A simple implementation of a Smart Hospital Management System."""
198     def __init__(self):
199         """Initialize the hospital management system with necessary data structures."""
200         self.patients = {} # Use a dictionary to store patient information
201         self.emergency_cases = [] # Use a list to manage emergency cases
202         self.medical_records = {} # Use a dictionary to store medical records
203         self.appointments = [] # Use a list to manage appointments
204         self.rooms = {} # Use a dictionary to manage hospital rooms
205
206     def check_in_patient(self, patient_id, patient_info):
207         """Check in a patient and store their information."""
208         self.patients[patient_id] = patient_info # Store patient information in the dictionary
209
210     def handle_emergency_case(self, case_info):
211         """Handle an emergency case by adding it to the emergency cases list."""
212         self.emergency_cases.append(case_info) # Add emergency case information to the list
213
214     def store_medical_record(self, patient_id, record):
215         """Store a medical record for a patient."""
216         if patient_id not in self.medical_records:
217             self.medical_records[patient_id] = [] # Initialize a list for the patient's medical records
218             self.medical_records[patient_id].append(record) # Add the medical record to the patient's list
219
220     def schedule_appointment(self, appointment_info):
221         """Schedule an appointment by adding it to the appointments list."""
222         self.appointments.append(appointment_info) # Add appointment information to the list
223
224     def navigate_to_room(self, room_number):
225         """Navigate to a hospital room by checking if it exists in the rooms dictionary."""
226         if room_number in self.rooms:
227             return f"Navigating to room {room_number}" # Return navigation message if room exists
228         else:
229             return f"Room {room_number} does not exist" # Return error message if room does not exist
230
231 # Example usage:
232 hospital = HospitalManagementSystem() # Create a new instance of the hospital management system
233 hospital.check_in_patient("P001", {"name": "John Doe", "age":
30}) # Check in a patient with ID P001
234 hospital.handle_emergency_case({"case_id": "E001", "description": "Heart Attack"}) # Handle an emergency case
235 hospital.store_medical_record("P001", {"record_id": "R001", "details
": "Blood P
236 hospital.schedule_appointment({"appointment_id": "A001", "patient_id": "P001", "date": "2024-07-01"}) # Schedule an appointment for pa
237 hospital.rooms["101"] = "General Ward" # Add a room to the hospital
238 print(hospital.navigate_to_room("101")) # Navigate to an existing room (Output: Navigating to room 101)
239 print(hospital.navigate_to_room("102")) # Navigate to a non-existing room (Output: Room 102 does not exist)
240
241

```

# table mapping feature → chosen data structure → justification.		
# Feature	Chosen Data Structure	Justification
# Patient Check-in	Dictionary	Allows for efficient storage and retrieval of patient information using unique patient IDs as keys.
# Emergency Case Handling	List	Maintains an ordered collection of emergency cases, allowing
# Medical Records Storage	Dictionary	Enables efficient storage and retrieval of medical records associated with patient IDs.
# Appointment Scheduling	List	Maintains an ordered collection of appointments, allowing for easy addition and retrieval.
# Hospital Room Navigation	Dictionary	Allows for efficient storage and retrieval of room information using room numbers as keys.

OUTPUT

```

PS C:\Users\hp\OneDrive\Desktop\AI_Assited_Coding> c;; cd
ugpy\launcher' '59565' '--' 'c:\Users\hp\OneDrive\Desktop\A
Navigating to room 101
Room 102 does not exist
PS C:\Users\hp\OneDrive\Desktop\AI_Assited_Coding>

```

JUSTIFICATION

Queue is suitable for patient check-in because patients are treated in the order they arrive (FIFO). This ensures fairness and systematic patient handling in hospitals. Queues are commonly used in service management systems such as hospitals, banks, and ticket counters.

Task 7 – Smart City Traffic Control System

Prompt

Design a Smart Traffic Management System by selecting suitable data structures for traffic signal queues, emergency vehicle handling, vehicle registration lookup, road network mapping, and parking slot tracking.

CODE

```
252
253 #Design a Smart Traffic Management System by selecting suitable data structures for traffic signal queues, emergency vehicle handling, vehicle registration lookup, road network
254 import numbers
255
256 class TrafficManagementSystem:
257     """A simple implementation of a Smart Traffic Management System."""
258     def __init__(self):
259         """Initialize the traffic management system with necessary data structures."""
260         self.traffic_signal_queues = {} # Use a dictionary to manage traffic signal queues
261         self.emergency_vehicle_queue = [] # Use a list to manage emergency vehicle queue
262         self.vehicle_registration_lookup = {} # Use a dictionary for vehicle registration lookup
263         self.road_network_mapping = {} # Use a dictionary to represent the road network
264         self.parking_slot_tracking = {} # Use a dictionary to track parking slots
265         # Additional methods for managing traffic signals, emergency vehicles, vehicle registration, road network, and parking slots would be implemented here
266
267 # Example usage:
268 traffic_system = TrafficManagementSystem() # Create a new instance of the traffic management system
269 # The following lines would demonstrate the usage of the traffic management system's features, such as managing
270
271 # traffic signal queues, handling emergency vehicles, looking up vehicle registrations, mapping the road network, and tracking parking slots.
272 # Table mapping feature + chosen data structure + justification.
273 # | Feature | Chosen Data Structure | Justification |
274 # |-----|-----|-----|
275 # | Traffic Signal Queues | Dictionary | Allows for efficient management of multiple traffic signal
276 # | Emergency Vehicle Handling | List | Maintains an ordered collection of emergency vehicles, allowing for easy addition and retrieval. |
277 # | Vehicle Registration Lookup | Dictionary | Enables efficient storage and retrieval of vehicle registration information using license plate numbers as keys. |
278 # | Road Network Mapping | Dictionary | Represents the road network as an adjacency list, allowing for efficient traversal and pathfinding. |
279 # | Parking Slot Tracking | Dictionary | Allows for efficient tracking of parking slot availability using slot numbers as keys. |
280
```

JUSTIFICATION

Queues are ideal for managing vehicles at traffic signals because vehicles pass in the order they arrive. This maintains smooth traffic flow and prevents unfair overtaking. Queue structures are commonly used in traffic systems and transportation management.

Task 8 – Smart E-Commerce Platform

Prompt

Design a Smart Online Shopping System by selecting suitable data structures for shopping cart management, order processing, product ranking, product search, and delivery route planning.

CODE

```

8
9 #Design a Smart Online Shopping System by selecting suitable data structures for shopping cart management, order processing, product ranking, product search, and delivery route planni
10
11 class OnlineShoppingSystem:
12     """A simple implementation of a Smart Online Shopping System."""
13
14     def __init__(self):
15         """Initialize the online shopping system with necessary data structures."""
16         self.shopping_cart = [] # Use a list to manage the shopping cart
17         self.orders = [] # Use a list to manage orders
18         self.product_ranking = {} # Use a dictionary to manage product rankings
19         self.product_search = {} # Use a dictionary to manage product search
20         self.delivery_route_planning = {} # Use a dictionary to manage delivery route planning
21
22         # Additional methods for managing the shopping cart, processing orders, ranking products, searching products, and planning delivery routes would be implemented here
23
24     # Example usage:
25     shopping_system = OnlineShoppingSystem() # Create a new instance of the online shopping system
26
27 # Table mapping feature + chosen data structure + justification.
28 # | Feature | Chosen Data Structure | Justification |
29 # |-----|-----|-----|
30 # | Shopping Cart Management | List | Maintains an ordered collection of items in the shopping cart, allowing for easy addition and removal of products. |
31 # | Order Processing | List | Maintains an ordered collection of orders, allowing for easy tracking and management of customer orders. |
32 # | Product Ranking | Dictionary | Enables efficient storage and retrieval of product rankings using product IDs as keys. |
33 # | Product Search | Dictionary | Allows for efficient search of products using product names or IDs as keys, enabling quick access to product information. |
34 # | Delivery Route Planning | Dictionary | Represents delivery routes as a graph, allowing for efficient traversal and optimization of delivery paths. |
35

```

JUSTIFICATION

Queue is suitable for order processing because orders should be processed in the same order they are placed (FIFO). This ensures fairness and proper order management. Queue-based systems are widely used in e-commerce platforms and transaction processing systems.